SASD GmbH

downloads-sasd

Developer Documentation and Extension Guide

Document purpose	**Explain how the code base is installed, understood and extended**
Project version	v0.1.0
Document version	v0.1.0
Prepared for	Robin Goerlach / SASD GmbH

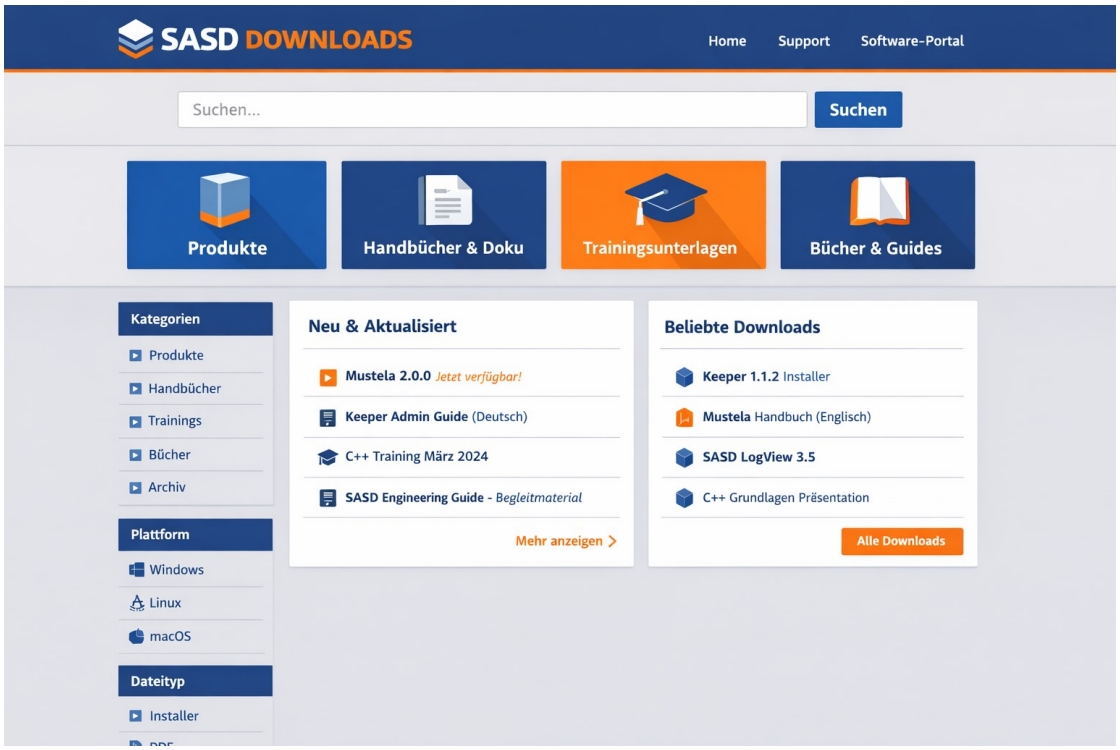

Figure 1 - Initial visual concept of the downloads.sasd.de home page.

1. Purpose, audience and document boundaries

This document is the technical developer handbook for **downloads-sasd**. It is intended to help a future maintainer set up the project, install optional Composer-based tooling, understand the internal structure, and extend the code base without breaking the original architectural direction.

The guide is written for a **single primary maintainer** scenario, but it is structured so that additional developers can onboard later with minimal guesswork. It complements the README, the setup guide and the administrator handbook rather than replacing them.

Reading recommendation
Read this document in the following order when returning to the project after a longer break: purpose and boundaries, local setup, application flow, repositories and catalog handling, then the extension workflow.

2. Technology profile and starting assumptions

The current implementation intentionally stays conservative. It uses **plain PHP 8.2**, a tiny custom bootstrapper, a small router, PHP views, and JSON-backed repositories. The code does not require a full framework and can run even without Composer during the first phase.

Area	Current choice	Reason today	Likely future evolution
Runtime	PHP 8.2	Simple hosting, low friction	Keep PHP, add packages where useful
Architecture	MVC-inspired layered structure	Readable, migration-friendly	Possibly stronger DI / service container later
Storage	JSON files	Quick to edit, versionable in Git	Database-backed repositories
Localization	PHP array language files	Easy to maintain	More locales and editorial tooling
Testing	Manual plus planned PHPUnit	Low initial overhead	Automated unit and integration tests

This is not a throwaway prototype. The file-based first version is deliberate because it keeps the first release small while preserving a clean migration path.

3. Preparing a development workstation

At minimum you need **PHP 8.2 or newer, a terminal, Git, a text editor or IDE**

Composer is **recommended** for installing development tooling, but the application bootstrap itself does not depend on Composer for the first version.

Tool	Mandatory now	Notes
PHP 8.2+	Yes	Required to run the application and CLI scripts.
Composer	Recommended	Useful for dev tooling such as PHPUnit and phpDocumentor.
Git	Yes	Needed for source control and safe catalog evolution.
LibreOffice or browser PDF viewer	Optional	Helpful when checking generated documentation.

3.1 Clone or create the project folder

```
$ git clone https://github.com/Robin-Goerlach/downloads-sasd.git
$ cd downloads-sasd

# If you start from a ZIP export, unpack it and switch into the project root.
```

3.2 Start the application without Composer

Because the project contains its own tiny PSR-4-like autoloader inside app/Bootstrap.php, you can start it immediately:

```
$ php -S 127.0.0.1:8080 -t public

# Open http://127.0.0.1:8080 in the browser.
```

Important boundary
The built-in web server is suitable for local development and quick verification only. Do not mistake it for a production deployment pattern.

4. Composer usage and recommended development packages

Composer is not required to boot the site, but it is the right place to manage **developer-only dependencies** such as tests, documentation generation and code-quality tooling. This lets the runtime stay lean while the developer experience improves.

4.1 Suggested Composer initialization strategy

```
$ composer install

# The current composer.json already defines the project name, PHP requirement and PSR-4 autoloading.
# If needed later, refresh the autoloader after adding new classes:
$ composer dump-autoload
```

4.2 Suggested dev packages

Package	Purpose	Recommended status	Typical command
phpunit/phpunit	Unit tests for services, repositories and helpers	Recommended	composer require --dev phpunit/phpunit
phpdocumentor/shim	Generate API documentation from PHPDoc blocks	Recommended	composer require --dev phpdocumentor/shim
phpstan/phpstan	Static analysis for early type and design issues	Optional but useful	composer require --dev phpstan/phpstan
friendsofphp/php-cs-fixer	Formatting and coding-style normalization	Optional	composer require --dev friendsofphp/php-cs-fixer

4.3 Example installation commands

```
$ composer require --dev phpunit/phpunit
$ composer require --dev phpdocumentor/shim
$ composer require --dev phpstan/phpstan
$ composer require --dev friendsofphp/php-cs-fixer

# Refresh autoload files after package changes
$ composer dump-autoload
```

4.4 Useful Composer scripts

The current project can profit from a few scripts in composer.json. A practical later extension would look like this:

```
"scripts": {
  "test": "phpunit",
  "docs:api": "phpdoc -d src -t build/phpdoc",
  "catalog:validate": "php cli/validate-catalog.php",
  "catalog:rebuild": "php cli/rebuild-catalog.php",
  "analyse": "phpstan analyse src app cli",
  "fix": "php-cs-fixer fix"
}
```

5. Repository layout and what each area is responsible for

Path	Responsibility
public/	Web entry point and public web root. index.php is the front controller.
app/	Bootstrap and application wiring.
src/Http/	Request, response, router and controllers.
src/Service/	Application logic, catalog operations and localization.
src/Repository/	Repository interfaces and JSON-backed implementations.
src/Infrastructure/	Lower-level catalog loading and filesystem concerns.
src/Domain/Entity/	Stable objects that represent products and artifacts.
src/View/ and resources/views/	HTML rendering support and templates.
resources/lang/	Translation array files for UI text.
config/	Application settings such as locales, brand colors and paths.
data/catalog/source/	Human-edited source JSON files.
data/catalog/generated/	Generated merged catalog file created by the rebuild script.
storage/files/	Physical downloadable files referenced by the catalog.
cli/	Catalog maintenance commands.
docs/	Project documentation assets such as screenshots and release notes.

6. Application bootstrap and request flow

The code path is intentionally short and explicit: **public/index.php** loads **app/Bootstrap.php**, the bootstrap resolves configuration paths, registers the small autoloader, builds the localization

layer, instantiates repositories and services, wires controllers, and returns a configured router plus request object.

```
public/index.php
  -> Bootstrap::boot()
     -> load config/app.php
     -> resolve paths
     -> Request::fromGlobals()
     -> LocaleResolver -> locale
     -> Translator
     -> CatalogLoader
     -> JsonProductRepository / JsonArtifactRepository
     -> CatalogService
     -> View
     -> Controllers
     -> Router route registration
  -> Router::dispatch(Request)
     -> Controller action
     -> Response object
  -> Response::send()
```

Design intent
The bootstrap currently performs explicit manual wiring instead of hiding dependencies inside a container. That is acceptable at this size and makes the application easier to understand for a solo maintainer.

6.1 Current route map

HTTP path	Controller action	Purpose
/	HomeController::index	Home page with search entry and highlighted content
/products	ProductController::index	Product overview
/products/{slug}	ProductController::show	Product details and related artifacts
/artifacts	ArtifactController::index	Artifact list and filtering entry point
/artifacts/{slug}	ArtifactController::show	Artifact details
/download/{slug}	ArtifactController::download	File delivery flow

7. Domain model and repository strategy

The most important architectural decision is not merely MVC; it is the **repository abstraction**. Controllers and services do not read JSON files directly. Instead, repositories return domain entities. This is what keeps the migration path toward a database realistic.

Layer	Key classes	Why it matters
Domain	Product, Artifact	Stable data structures used by the application.
Repository	ProductRepositoryInterface, ArtifactRepositoryInterface, Json*Repository	Separates data access from business logic.
Infrastructure	CatalogLoader	Handles JSON reading, validation and generated catalog access.

7.1 Why JSON access must stay inside repositories

1. It keeps controllers small and focused on HTTP concerns.
2. It prevents duplicate file-reading code from spreading across the project.
3. It lets you swap the storage backend later by replacing repository implementations.
4. It makes unit testing easier because services can be tested against interfaces or stubs.

8. Catalog source files and generated catalog workflow

The project distinguishes between human-edited source data and a generated merged catalog:

Location	Edited by human	Purpose
data/catalog/source/products.json	Yes	Primary product definitions
data/catalog/source/artifacts.json	Yes	Primary artifact definitions
data/catalog/generated/catalog.json	No, generated	Merged file read preferentially at runtime if available

The **validate** command checks for required fields and missing files. The **rebuild** command writes a consolidated catalog. The application prefers the generated file when it exists.

```
$ php cli/validate-catalog.php
```

```
$ php cli/rebuild-catalog.php
```

8.1 Safe editing sequence for catalog changes

1. Create or update the physical file under storage/files/ first.
2. Edit the relevant JSON source entry with product and artifact metadata.
3. Run catalog validation and fix every error before proceeding.
4. Rebuild the generated catalog.
5. Start the application locally and verify the affected pages.
6. Only then commit the change to Git.

9. Internationalization design

The project already supports **German and English** and is prepared for additional languages. UI labels live in **resources/lang/*.php**, while business content such as product titles and artifact descriptions is localized inside the JSON catalog entries.

Concern	Current implementation	Extension pattern
Locale resolution	LocaleResolver chooses from enabled locales and fallback locale	Add new locale to config/app.php
UI strings	Translator reads PHP array files	Create resources/lang/<locale>.php
Catalog content	Localized title/summary/description arrays in JSON	Add matching locale keys in product and artifact records

9.1 Recommended language policy for future growth

- Keep locale codes short and standardised; for Hindi use hi, not a generic word like 'Indian'.
- For Chinese, be ready to split generic zh into region-specific variants later, such as zh-CN and zh-TW.
- Treat English as the fallback content source unless a different fallback strategy becomes necessary.
- Do not mix translated UI labels with untranslated catalog records in a release without consciously accepting that result.

10. Testing, documentation generation and quality checks

The repository starts lean, but **quality automation should grow with the code base**. PHPUnit and phpDocumentor are the first two packages worth adding because they support confidence and maintainability without dragging in heavy infrastructure.

10.1 PHPUnit recommendation

Start with focused tests around service and repository behaviour before attempting broad UI tests.

Good first test targets	What to verify
CatalogService	Search behaviour, filtering, group counts, featured/popular ordering.
LocaleResolver	Locale selection from query string and fallback behaviour.
Translator	Translation lookup and fallback key resolution.
CatalogLoader	Validation of required fields and missing files.
Json repositories	Correct creation of Product and Artifact entities.

10.2 Example PHPUnit skeleton

```php
<?php
declare(strict_types=1);

use PHPUnit\Framework\TestCase;
use App\Service\Localization\Translator;

final class TranslatorTest extends TestCase
{
    public function testFallbackLocaleIsUsedWhenKeyIsMissing(): void
    {
        // Arrange, act and assert here.
        $this->assertTrue(true);
    }
}
```

10.3 phpDocumentor recommendation

The code is already written with PHPDoc-style comments in mind. Generating API documentation later should be straightforward:

```
$ vendor/bin/phpdoc -d src -t build/phpdoc
```

```
# Or, if the shim package exposes phpdoc directly:
$ phpdoc -d src -t build/phpdoc
```

Documentation rule
Keep PHPDoc meaningful. Document purpose, parameters, return shapes and invariants. Avoid filler comments that merely repeat the method name in prose.

11. How to understand the code quickly after a break

1. Read config/app.php to remember the supported locales, brand values and path assumptions.
2. Read app/Bootstrap.php to see how everything is wired together.
3. Inspect Router.php and the controller classes to recall the public entry points.
4. Read CatalogService before changing behaviour, because this is where presentation-oriented logic is concentrated.
5. Read CatalogLoader and the JSON repositories before touching storage or migration-related code.
6. Open the JSON source files and one real artifact file to reconnect the code to actual data.

12. Recommended workflow for adding a new feature

Step	Action	Reason
1	Clarify whether the change belongs to HTTP, service, repository, infrastructure or view layer.	Prevents logic from being placed in the wrong class.
2	Define or confirm the user-visible result first.	Keeps architecture tied to a concrete need.
3	Extend domain and repository contracts before touching controllers if data access changes.	Preserves separation of concerns.
4	Implement the smallest service change that satisfies the requirement.	Keeps behaviour centralised.
5	Update or add tests and run	Reduces regression risk.

	catalog validation if content is involved.	
6	Verify in the browser and check both German and English output if visible text changed.	Protects the multilingual baseline.

12.1 Typical examples

- New filter on the artifacts page: extend CatalogService and the controller input handling, then update the view.
- Database migration: add new DbProductRepository and DbArtifactRepository implementations, then adjust Bootstrap wiring.
- New UI language: add a new language file, activate the locale in config/app.php and add translated catalog data.
- Download logging: introduce a dedicated service or infrastructure component instead of inserting logging code directly into the controller.

13. Coding and commenting conventions for this project

The project should favour **clear names, generous comments where they help understanding, and disciplined layering**. In practice that means:

- Use /** ... */ comments for class, method and non-trivial property documentation.
- Use // comments for local intent, temporary clarifications or step-by-step reasoning inside a method.
- Do not comment obvious one-line assignments unless there is a design reason behind them.
- Prefer explicit value objects and method names over cryptic abbreviations.
- When a trade-off is intentional, document the reason once near the decision point.

13.1 Example of useful internal commentary

```
/**
 * Builds the application and returns the configured router together
 * with the request instance. Composer stays optional in v0.1.0, so
 * a tiny internal autoloader is registered here as well.
 */
public static function boot(string $basePath): array
{
    // Paths are resolved once here so downstream code can rely on
    // absolute filesystem paths instead of repeating join logic.
    ...
}
```

14. Troubleshooting and common developer mistakes

Symptom	Likely cause	Recommended check
404 for a page that should exist	Route not registered or wrong slug	Check Router registration and the JSON slug values.
Artifact page exists but download fails	Missing file in storage/files or bad path in artifacts.json	Run php cli/validate-catalog.php and inspect the path field.
English page still shows German content	English translation keys or catalog text missing	Check resources/lang/en.php and the localized JSON content.
Code class not found	New PHP file added but namespace/path mismatch	Check PSR-4 path under src/ and run composer dump-autoload if Composer is in use.
Catalog changes do not appear	Generated catalog still stale	Run the rebuild command and refresh the browser.

15. Recommended next engineering steps

- Add Composer dev dependencies and script shortcuts.
- Introduce a basic PHPUnit suite for CatalogService, Translator and CatalogLoader.
- Add a phpDocumentor target so the API surface can be browsed quickly.
- Create a small CONTRIBUTING.md once another contributor joins the project.
- Introduce CI checks for syntax, tests and catalog validation before public releases.
- Keep the current architecture small; avoid framework inflation before the project truly needs it.

16. Final orientation

When in doubt, protect the **repository abstraction, small service layer, clear paths, predictable JSON workflow, multilingual discipline**. Those decisions are what make the current version both practical today and extensible tomorrow.